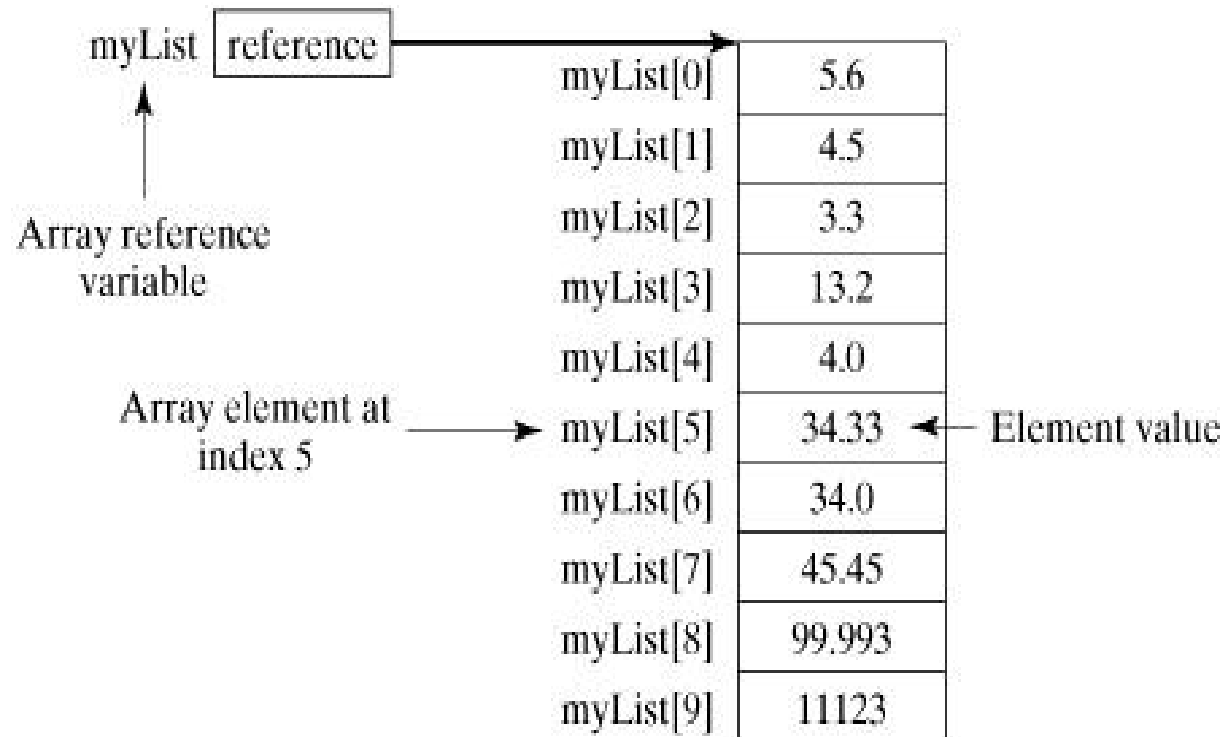


Array

- An array is used to store a collection of data of the same type.
- Instead of declaring individual variables, such as number0, number1, ..., and number99, you declare one array variable such as numbers and use numbers[0], numbers[1], and ..., numbers[99] to represent individual variables.
- Declaring Array Variables:
dataType[] arrayRefVar;
dataType arrayRefVar[];
- Example:
 double[] myList;
 double myList[];
- Example :
 double[] myList = new double[10];



- **One- Dimensional Arrays**
- *A one-dimensional array is, essentially, a list of like-typed variables. To create an array, you first must create an array variable of the desired type. The general form of a one-dimensional array declaration is*

type var-name[]; //One-Dimensional Arrays

- **Example:**

`int month_days[];`

Allocate memory to array variable.

array-var = new type[size];

- **Example**

`month_days = new int[12];`

- Once you have allocated an array, you can access a specific element in the array by specifying its index within square brackets. All array indexes start at zero. For example, this statement assigns the value 28 to the second element of **month_days**.

`month_days[1] = 28;`

- The next line displays the value stored at index 3.

`System.out.println(month_days[3]);`

```
class Array
{
    public static void main(String args[])
    {
        int month_days[];
        month_days = new int[12];
        month_days[0] = 31;
        month_days[1] = 28;
        month_days[2] = 31;
        month_days[3] = 30;
        month_days[4] = 31;
        month_days[5] = 30;
        month_days[6] = 31;
        month_days[7] = 31;
        month_days[8] = 30;
        month_days[9] = 31;
        month_days[10] = 30;
        month_days[11] = 31;
        System.out.println("April has " + month_days[3] + " days.");
    }
}
```

- It is possible to combine the declaration of the array variable with the allocation of the array itself, as shown here:

```
int month_days[] = new int[12];
```
- Arrays can be initialized when they are declared. The process is much the same as that used to initialize the simple types.
- For example, to store the number of days in each month, the following code creates an initialized array of integers:

```
class AutoArray
{
    public static void main(String args[])
    {
        int month_days[] = { 31, 28, 31, 30, 31, 30, 31, 31, 30, 31,
                             30, 31 };
        System.out.println("April has " + month_days[3] + " days.");
    }
}
```

- Java strictly checks to make sure you do not accidentally try to store or reference values outside of the range of the array.
- For example, the run-time system will check the value of each index into **month_days to make sure that it is between 0 and 11 inclusive. If you try to** access elements outside the range of the array (negative numbers or numbers greater than the length of the array), you will cause a run-time error.

- **Alternative Array Declaration Syntax**
- There is a second form that may be used to declare an array:
`type[ ] var-name;`
- Here, the square brackets follow the type specifier, and not the name of the array variable. For example, the following two declarations are equivalent:
`int al[] = new int[3];`
`int[] a2 = new int[3];`
- The following declarations are also equivalent:
`char twod1[][] = new char[3][4];`
`char[][] twod2 = new char[3][4];`
- This alternative declaration form offers convenience when declaring several arrays at the same time. For example,
`int[] nums, nums2, nums3; // create three arrays`
- creates three array variables of type **int**. **It is the same as writing**
`int nums[], nums2[], nums3[]; // create three arrays`

```
class Length
{
    public static void main(String args[])
    {
        int a1[] = new int[10];
        int a2[] = {3, 5, 7, 1, 8, 99, 44, -10};
        int a3[] = {4, 3, 2, 1};
        System.out.println("length of a1 is " + a1.length);
        System.out.println("length of a2 is " + a2.length);
        System.out.println("length of a3 is " + a3.length);
    }
}
```

This program displays the following output:

length of a1 is 10

length of a2 is 8

length of a3 is 4

String Handling in JAVA

- **String Handling** provides a lot of concepts that can be performed on a string such as concatenating string, comparing string, substring etc.
- In java, string is basically an immutable object.
- Generally string is a sequence of characters. But in java, string is an object.
- ***String class*** is used to create string object.

```
class Simple
{
    public static void main(String args[])
    {
        String s="Sachin";
        s=s.concat(" Tendulkar");
        System.out.println(s);
    }
}
```

String comparison in Java

- We can compare two given strings on the basis of content and reference.
- It is used in **authentication** (by equals() method), **sorting** (by compareTo() method), **reference matching** (by == operator) etc.
- There are three ways to compare String objects:
 - By equals() method
 - By == operator
 - By compareTo() method

1) By equals() method

- equals() method compares the original content of the string. It compares values of string for equality. String class provides two methods:
 - **public boolean equals(Object another){}** compares this string to the specified object.
 - **public boolean equalsIgnoreCase(String another){}** compares this String to another String, ignoring case.

```
class Simple{  
    public static void main(String args[]){  
  
        String s1="Sachin";  
        String s2="Sachin";  
        String s3=new String("Sachin");  
        String s4="Saurav";  
  
        System.out.println(s1.equals(s2));//true  
        System.out.println(s1.equals(s3));//true  
        System.out.println(s1.equals(s4));//false  
    }  
}
```

//Example of equalsIgnoreCase(String) method

class Simple{

public static void main(String args[]){

String s1="Sachin";

String s2="SACHIN";

System.out.println(s1.equals(s2));//false

System.out.println(s1.equalsIgnoreCase(s3));//true

}

}

By compareTo() method:

- compareTo() method compares values and returns an int which tells if the values compare less than, equal, or greater than. Suppose s1 and s2 are two string variables. If:
 - **s1 == s2** :0
 - **s1 > s2** :positive value
 - **s1 < s2** :negative value

```
class Simple{  
    public static void main(String args[]){  
  
        String s1="Sachin";  
        String s2="Sachin";  
        String s3="Ratan";  
  
        System.out.println(s1.compareTo(s2));//0  
        System.out.println(s1.compareTo(s3));//1(because s1>s3)  
        System.out.println(s3.compareTo(s1));//1(because s3 < s1  
        )  
    }  
}
```

String Concatenation in Java

- Concating strings form a new string i.e. the combination of multiple strings.
- There are two ways to concat string objects:
- By + (string concatenation) operator
- By concat() method
- **By + (string concatenation) operator**
- String concatenation operator is used to add strings.For Example:

```
class Simple{  
    public static void main(String args[]){  
  
        String s="Sachin"+" Tendulkar";  
        System.out.println(s);//Sachin Tendulkar  
    }  
}
```

By concat() method

- concat() method concatenates the specified string to the end of current string.
- **Syntax:** public String concat(String another){}

```
class Simple{
```

```
    public static void main(String args[]){
```

```
        String s1="Sachin ";
```

```
        String s2="Tendulkar";
```

```
        String s3=s1.concat(s2);
```

```
        System.out.println(s3);//Sachin Tendulkar
```

```
    }
```

```
}
```

Substring in Java

- A part of string is called **substring**. In other words, substring is a subset of another string.
- In case of substring `startIndex` starts from 0 and `endIndex` starts from 1 or `startIndex` is inclusive and `endIndex` is exclusive.
- You can get substring from the given String object by one of the two methods:
- **`public String substring(int startIndex)`**: This method returns new String object containing the substring of the given string from specified `startIndex` (inclusive).
- **`public String substring(int startIndex, int endIndex)`**: This method returns new String object containing the substring of the given string from specified `startIndex` to `endIndex`.

0 1 2 3 4
| | | | |
HELLO
| | | | |
1 2 3 4 5

Start Index starts from 0

End Index starts from 1

```
class Simple{  
    public static void main(String args[]){  
  
        String s="Sachin Tendulkar";  
        System.out.println(s.substring(6));//Tendulkar  
        System.out.println(s.substring(0,6));//Sachin  
    }  
}
```

Methods of String class in Java

- `java.lang.String` class provides a lot of methods to work on string. By the help of these methods, we can perform operations on string such as trimming, concatenating, converting strings etc.

toUpperCase() and toLowerCase() method

- public String toUpperCase()
 - Converts all of the characters in this String to upper case
- public String toLowerCase()
 - Converts all of the characters in this String to lower case.

```
class Simple{
```

```
    public static void main(String args[]){
```

```
        String s="Sachin";
```

```
        System.out.println(s.toUpperCase());//SACHIN
```

```
        System.out.println(s.toLowerCase());//sachin
```

```
        System.out.println(s);//Sachin(no change in original)
```

```
    }
```

```
}
```

trim() method

- `public String trim()`
 - Returns a copy of the string, with leading and trailing whitespace omitted.

```
class Simple{  
    public static void main(String args[]){
```

```
        String s=" Sachin ";  
        System.out.println(s);// Sachin  
        System.out.println(s.trim());//Sachin
```

```
    }
```

```
}
```

charAt() method & length() method

- public char charAt(int index)
 - Returns the char value at the specified index.
- public int length()
 - Returns the length of this string.

```
class Simple{  
    public static void main(String args[]){
```

```
        String s="Sachin";  
        System.out.println(s.charAt(0));//S  
        System.out.println(s.charAt(3));//h  
        System.out.println(s.length());//6  
    }  
}
```